Tight Security of XMSS in the ROM

Work in Progress

Abstract

We present a classical, almost tight, multi-user, proof of security of XMSS (eXtended Merkle Signature Scheme) against *strong unforgeability under chosen message attack* (SUF-CMA), in the random oracle model.

Contents

1 Introduction

XMSS is a hash-based, stateful, signature scheme that achieves security based solely on the properties of the underlying hash function. In this work, we analyze its security in the random oracle model, proving strong unforgeability under chosen message attacks (SUF-CMA) in a multi-user setting.

Our analysis proceeds by classifying all possible forgery attacks into four (exhaustive) categories (E1–E4), then bounding the probability of each. The main technical challenges involve carefully tracking the adversary's information throughout the security game using a tree-based analysis.

At the cost of relying on the Random Oracle Model, our proof achieves almost tight bounds on the adversary's success probability, bringing practical improvements upon [1], [2], [3] (that rely purely on standard model assumptions).

2 Definitions and Parameters

We begin by establishing the notation and parameters used throughout this analysis.

2.1 Basic Parameters

- n: Number of bits in a hash digest.
- $H : \{0,1\}^* \to \{0,1\}^n$: A hash function modeled as a (lazy) random oracle.
- w: Hash chain length in WOTS (Winternitz One-Time Signature), assumed to be a power of 2.
- v: Number of hash chains in WOTS.
- pp: Number of bits for the public parameter—a random salt included in the XMSS public key and used in all hash function calls as protection against multi-user attacks.
- L: Lifetime, defined as the total number of signatures that can be generated with each XMSS key pair. The Merkle tree has height $h = \log_2(L)$, with L leaves indexed by an "epoch" ranging from 0 to $L - 1$.
- t: Target sum for the encoding scheme.
- r: Number of bits of randomness the signer generates before encoding.
- N: Number of signers in the multi-user setting.
- q: Total number of random oracle (H) queries made by the adversary.

2.2 Domain Separation Constants

Definition 2.1 (Domain Separator Constants)**.** The scheme uses four domain separator values to distinguish hash contexts:

- $\mathsf{D_{TREE}} = 0$: Hash computations for internal Merkle tree nodes.
- $\mathsf{D_{CHAIN}} = 1$: Hash computations within WOTS hash chains.
- $\mathsf{D_{ENC}} = 2$: Hash computations for message encoding.
- $\mathsf{D_{LEAF}} = 3$: Hash computations for WOTS public key compression (Merkle tree leaves).

2.3 XMSS Public Key Structure

The XMSS public key consists of two components:

$$\mathsf{pk}_{\mathrm{XMSS}} = (\mathsf{PP}, \mathsf{root})$$

where $\mathsf{PP} \in \{0,1\}^{\mathsf{pp}}$ is the public parameter (sampled uniformly at random during key generation) and $\mathsf{root} \in \{0,1\}^n$ is the Merkle tree root. Each leaf of the Merkle tree corresponds to the hash of a WOTS public key.

2.4 Tweaked Hash Functions

Each invocation of the hash function H incorporates the public parameter PP alongside a *tweak*—which includes the domain separator and additional indices that uniquely identify the position of the hash computation within the scheme.

- **Merkle tree nodes:** The tweak contains $\mathsf{D}_{\mathsf{TREE}}$, the depth d, and node index i.
- **WOTS chains:** The tweak contains $\mathsf{D}_{\mathsf{CHAIN}}$, the leaf index (epoch e), chain index j, and position k within the chain.
- **Message encoding:** The tweak contains $\mathsf{D}_{\mathsf{ENC}}$, the epoch e, randomness R, and message m.
- **WOTS public key compression:** The tweak contains $\mathsf{D}_{\mathsf{LEAF}}$, the tree height h, and the leaf index i (i.e., the epoch).

Definition 2.2 (Epoch)**.** The *epoch* e denotes the index of the WOTS key pair within the Merkle tree, equivalently the leaf index. Valid epochs satisfy $0 \leq e < L$.

3 Scheme Description

3.1 Message Encoding

To sign a message $m \in \{0,1\}^{256}$ at epoch $e < L$, the signer performs the following encoding procedure.

1. Sample randomness $R \in \{0,1\}^r$ uniformly at random.
2. Compute $H(\mathsf{PP} \| \mathsf{D}_{\mathsf{ENC}} \| e \| R \| m)$ and interpret the first $v \cdot \log_2 w$ bits as a sequence of v integers $(x_0, x_1, \ldots, x_{v-1})$ where each $x_i \in \{0, 1, \ldots, w-1\}$.
3. Check whether $\sum_{i=0}^{v-1} x_i = t$ for a fixed target sum t (typically $t = v(w-1)/2$).
4. If the constraint is not satisfied, resample R and repeat from step 1.

Definition 3.1 (Encoding Probability)**.** Let P_{enc} denote the probability that a uniformly random sequence of v values from $\{0, 1, \ldots, w-1\}$ satisfies the target sum constraint $\sum_{i=0}^{v-1} x_i = t$. On average, generating valid randomness requires $1/P_{\mathrm{enc}}$ attempts.

Remark 3.2 (Incomparability of Encodings). The target sum constraint ensures that valid encodings are *incomparable*: given two *different* valid encodings $(x_0, \ldots, x_{v-1})$ and $(x'_0, \ldots, x'_{v-1})$, there must exist indices j and k such that $x_j < x'_j$ and $x_k > x'_k$.

3.2 WOTS Signature Scheme

3.2.1 Key Generation

For epoch e:

- **Secret key:** $\mathsf{sk} = (\mathsf{sk}_0, \mathsf{sk}_1, \ldots, \mathsf{sk}_{v-1})$ where each $\mathsf{sk}_i \in \{0,1\}^n$ is chosen uniformly at random.
- **Public key:** $\mathsf{pk} = (\mathsf{pk}_0, \mathsf{pk}_1, \ldots, \mathsf{pk}_{v-1})$ where

$$\mathsf{pk}_i = H^{w-1}_{\mathsf{PP},\mathsf{D}_{\mathsf{CHAIN}},e,i}(\mathsf{sk}_i)$$

  denotes sk_i hashed $w-1$ times with tweaks indicating the domain separator $\mathsf{D}_{\mathsf{CHAIN}}$, epoch e, chain index i, and positions $k \in \{0,1,\ldots,w-2\}$.

Explicitly, we define $c_{i,0} := \mathsf{sk}_i$ and for $k = 0,1,\ldots,w-2$:

$$c_{i,k+1} := H(\mathsf{PP} \| \mathsf{D}_{\mathsf{CHAIN}} \| e \| i \| k \| c_{i,k})$$

so that $\mathsf{pk}_i = c_{i,w-1}$. We call $c_{i,k}$ the *chain value* at position k in chain i.

3.2.2 Signature Generation

For a message with encoding $(x_0, x_1, \ldots, x_{v-1})$:

$$\mathsf{sig} = (\mathsf{sig}_0, \mathsf{sig}_1, \ldots, \mathsf{sig}_{v-1}) \quad \text{where} \quad \mathsf{sig}_i = c_{i,x_i}$$

3.2.3 Signature Verification

Given signature sig and encoding $(x_0, x_1, \ldots, x_{v-1})$, the verifier reconstructs the WOTS public key: for each $i \in \{0,1,\ldots,v-1\}$, compute

$$\mathsf{pk}'_i = H^{w-1-x_i}_{\mathsf{PP},\mathsf{D}_{\mathsf{CHAIN}},e,i}(\mathsf{sig}_i)$$

by applying $w-1-x_i$ hash iterations starting from position x_i.

3.3 Merkle Tree Construction

3.3.1 Leaf Computation

The i-th leaf of the Merkle tree is the hash of the corresponding WOTS public key:

$$\mathsf{leaf}_i = H(\mathsf{PP} \| \mathsf{D}_{\mathsf{LEAF}} \| i \| \mathsf{pk}_0 \| \mathsf{pk}_1 \| \cdots \| \mathsf{pk}_{v-1})$$

3.3.2 Internal Node Computation

For sibling nodes A and B (both in $\{0,1\}^n$) at positions $2i$ and $2i+1$ at depth d (where depth h is the leaf level, and depth 0 is the root):

$$\mathsf{parent} = H(\mathsf{PP} \| \mathsf{D}_{\mathsf{TREE}} \| d \| i \| A \| B)$$

3.4 XMSS Signature Structure

An XMSS signature for message $m \in \{0,1\}^{256}$ at epoch e consists of:

1. Randomness $R \in \{0,1\}^r$ used for encoding.
2. WOTS signature $\mathsf{sig} = (\mathsf{sig}_0, \mathsf{sig}_1, \ldots, \mathsf{sig}_{v-1})$.
3. Authentication path $(\mathsf{auth}_0, \mathsf{auth}_1, \ldots, \mathsf{auth}_{h-1})$ consisting of h sibling node digests.

Verification proceeds by: (1) recomputing the encoding from R and m, (2) reconstructing the WOTS public key from sig, (3) computing the leaf hash, and (4) using the authentication path to compute a candidate root, which is compared against the public root.

4 Security Model

We prove security in the *strong unforgeability under chosen message attack* (SUF-CMA) model in a multi-user setting.

Definition 4.1 (Multi-User SUF-CMA Security Game)**.** Let $\Sigma = (\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$ be a stateful signature scheme and let $N \in \mathbb{N}$ be the number of signers. The multi-user SUF-CMA game between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$ proceeds as follows:

1. **Setup.** The challenger generates N independent key pairs:

$$(\mathsf{pk}_i, \mathsf{sk}_i) \leftarrow \mathsf{KeyGen}(1^\lambda) \quad \text{for } i \in \{0, \ldots, N-1\}$$

   and gives $(\mathsf{pk}_0, \ldots, \mathsf{pk}_{N-1})$ to the adversary. The challenger initializes $\mathcal{Q}_{i,e} \leftarrow \emptyset$ for all $i \in \{0, \ldots, N-1\}$ and $e \in \{0, \ldots, L-1\}$.

2. **Query phase.** The adversary adaptively interleaves the following queries:

   - *Random oracle query:* $\mathcal{A}$ queries $H(x)$ for any input $x \in \{0,1\}^*$ and receives the output.
   - *Signing query:* $\mathcal{A}$ queries (i, e, m) where $i \in \{0, \ldots, N-1\}$, $e \in \{0, \ldots, L-1\}$, and $m \in \{0,1\}^{256}$. If $\mathcal{Q}_{i,e} \neq \emptyset$ (epoch already used for signer i), the query is rejected (and the game aborts). Otherwise, the challenger computes $\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}_i, e, m)$, sets $\mathcal{Q}_{i,e} \leftarrow \{(m, \sigma)\}$, and returns σ to $\mathcal{A}$.

     Remark 4.2. When computing the signature, the challenger iteratively picks at random, uniformly and without repetitions, $R \in \{0,1\}^r$ until a valid encoding is found. If after all the 2^r choices of R no valid encoding is found, the game aborts.

3. **Forgery.** The adversary outputs (i, e, m^*, σ^*).

4. **Winning condition.** The adversary *wins* if:

   (a) $\mathsf{Verify}(\mathsf{pk}_i, e, m^*, \sigma^*) = 1$, and

   (b) $(m^*, \sigma^*) \notin \mathcal{Q}_{i,e}$.

Remark 4.3 (Strong Unforgeability). Condition (b) captures *strong* unforgeability: the adversary wins even if m^* was previously signed at epoch e by signer i, provided σ^* differs from the signature previously returned. This is stronger than standard unforgeability, which only requires the message to be new.

Remark 4.4 (Maximum Signatures). The total number of signatures across all signers is at most $L \cdot N$.

4.1 Adversary Model

We consider a classical (non-quantum) adversary. We model the adversary as a deterministic algorithm; this assumption is without loss of generality in the random oracle model, as any randomized adversary can be derandomized by fixing its random tape to its most successful value.

We assume without loss of generality that the adversary requests the maximum number of signatures ($L \cdot N$ in total), as requesting fewer signatures can only reduce the adversary's advantage.

We also assume without loss of generality that the adversary's requests are all valid (i.e., no signer is queried at the same epoch twice), and the adversary never performs the same random oracle query more than once.

5 Security Analysis

Let q denote the total number of (classical) random oracle queries made by the adversary.

5.1 Public Parameter Collisions

Proposition 5.1. *Except with probability* $\Pr[PP\ collision] \leq N^2/2^{\mathsf{pp}+1}$*, all public parameters across the* N *signers are distinct.*

Proof. Each signer samples $\mathsf{PP} \in \{0,1\}^{\mathsf{pp}}$ independently and uniformly at random. By Theorem A.1 with $k = N$ values sampled from a set of size 2^{pp}, the probability of any collision is at most $N^2/(2 \cdot 2^{\mathsf{pp}}) = N^2/2^{\mathsf{pp}+1}$. □

Assumption: For the remainder of the analysis, we condition on the event that all public parameters are distinct.

5.2 Classification of Forgery Events

Suppose the adversary produces a valid forgery:

$$\sigma^* = (R^*, (\mathsf{sig}_0^*, \ldots, \mathsf{sig}_{v-1}^*), (\mathsf{auth}_0^*, \ldots, \mathsf{auth}_{h-1}^*))$$

for signer i, at epoch e, on message m^*.

Let (m, σ) with $\sigma = (R, (\mathsf{sig}_0, \ldots, \mathsf{sig}_{v-1}), (\mathsf{auth}_0, \ldots, \mathsf{auth}_{h-1}))$ denote the corresponding message and (authentic) signature at epoch e for signer i. Let $(x_0, \ldots, x_{v-1})$ and $(x_0^*, \ldots, x_{v-1}^*)$ be the encodings derived from (R, m) and (R^*, m^*), respectively. Similarly, let leaf and leaf^* denote the corresponding leaf hashes.

Since $(m^*, \sigma^*) \neq (m, \sigma)$, at least one of the following events must occur:

E1 (Encoding Collision): $(R^*, m^*) \neq (R, m)$ but the encodings are identical, i.e., the first $v \cdot \log_2(w)$ bits of $H(\mathsf{PP} \| \mathsf{D}_{\mathsf{ENC}} \| e \| R^* \| m^*)$ equal those of $H(\mathsf{PP} \| \mathsf{D}_{\mathsf{ENC}} \| e \| R \| m)$.

E2 (WOTS Chain Collision): The forged WOTS public key is the same as the authentic one: $(\mathsf{pk}_0^*, \ldots, \mathsf{pk}_{v-1}^*) = (\mathsf{pk}_0, \ldots, \mathsf{pk}_{v-1})$, but the WOTS signature components differ: $(x_0^*, \ldots, x_{v-1}^*, \mathsf{sig}_0^*, \ldots, \mathsf{sig}_{v-1}^*) \neq (x_0, \ldots, x_{v-1}, \mathsf{sig}_0, \ldots, \mathsf{sig}_{v-1})$.

E3 (Leaf Collision): The WOTS public keys differ: $(\mathsf{pk}_0^*, \ldots, \mathsf{pk}_{v-1}^*) \neq (\mathsf{pk}_0, \ldots, \mathsf{pk}_{v-1})$, but their hashes collide: $H(\mathsf{PP} \| \mathsf{D}_{\mathsf{LEAF}} \| e \| \mathsf{pk}_0 \| \mathsf{pk}_1 \| \cdots \| \mathsf{pk}_{v-1}) = H(\mathsf{PP} \| \mathsf{D}_{\mathsf{LEAF}} \| e \| \mathsf{pk}_0^* \| \mathsf{pk}_1^* \| \cdots \| \mathsf{pk}_{v-1}^*)$.

E4 (Merkle Tree Collision): Among the forged Merkle path, there exist two siblings (A^*, B^*), different from the authentic siblings (A, B) at the corresponding position, such that their hashes collide: $H(\mathsf{PP} \| \mathsf{D}_{\mathsf{TREE}} \| d \| i \| A \| B) = H(\mathsf{PP} \| \mathsf{D}_{\mathsf{TREE}} \| d \| i \| A^* \| B^*)$.

Proposition 5.2 (Exhaustiveness)**.** *If the adversary wins the SUF-CMA game, then at least one of E1, E2, E3, or E4 occurs.*

Proof. Assume by contradiction that none of E1, E2, E3, or E4 occurs.

Identical Merkle path:

Since $E4$ does not occur, the authentication path nodes must be identical:

$$(\mathsf{auth}_0^*, \ldots, \mathsf{auth}_{h-1}^*) = (\mathsf{auth}_0, \ldots, \mathsf{auth}_{h-1})$$

Identical WOTS Public Keys:

Suppose for contradiction that the WOTS public keys differ. Then since E3 does not occur, the leaf hashes must differ: $\mathsf{leaf}^* \neq \mathsf{leaf}$. Starting from these different leaves at depth h, both

authentication paths lead to the same root (since verification succeeds for both). Let d^* be the smallest depth at which the computed nodes first coincide. At depth $d^* + 1$, the nodes differ but hash to the same parent at depth d^*—this is precisely E4, contradicting our assumption.

Identical WOTS signature:

Since the WOTS public keys are identical, and E2 does not occur, we must have:

$$(x_0^*, \ldots, x_{v-1}^*, \mathsf{sig}_0^*, \ldots, \mathsf{sig}_{v-1}^*) = (x_0, \ldots, x_{v-1}, \mathsf{sig}_0, \ldots, \mathsf{sig}_{v-1})$$

Identical Randomness and Message:

Since the encodings are identical, and E1 does not occur, we must have $(R^*, m^*) = (R, m)$.

Contradiction:

We have shown that both the message m^* and the signature σ^* are identical to the authentic ones (m, σ), contradicting the assumption that the adversary wins the SUF-CMA game.

□

5.3 Analysis Framework

We represent the execution of the game as a tree where:

- Each node represents a query from the adversary: either a *random oracle query* or a *signature request*.
- From a *random oracle query node*, there are 2^n uniformly distributed children (one per possible output) by a prior signature request, or by the initial key generation, in which case there is only one child (with probability 1).
- From a *signature request node*, the branching reflects the challenger's search for valid randomness. The challenger samples $R \in \{0,1\}^r$ uniformly at random without replacement until finding one that yields a valid encoding. For each candidate R:
  - If the adversary has previously queried $H(\mathsf{PP} \| \mathsf{D}_{\mathsf{ENC}} \| e \| R \| m)$, the encoding is already determined (either valid or invalid), contributing no additional randomness.
  - If the adversary has not queried this value, the hash output is sampled uniformly, yielding a valid encoding with probability P_{enc}.

  Additionally, there is one child corresponding to the event that no valid encoding exists among all 2^r choices of R. This occurs with negligible probability (bounded by $(1-P_{\text{enc}})^{2^r}$), and we will neglect it in the analysis.

At the end of each branch of the tree is associated the final answer of the adversary: (i, e, m^*, σ^*).

A random execution corresponds to a random path from root to leaf: we are interested in bounding the probability that the final answer corresponds to a successful forgery.

6 Event E1: Encoding Collision

We consider the simpler game E1' in which the attacker wins if at some point in the game, there exists [1] a previous random oracle query $(\mathsf{PP} \| \mathsf{D}_{\mathsf{ENC}} \| e \| R^* \| m^*)$ and a previous signature request (i, e, m), with public parameter PP and randomness R, such that $(R^*, m^*) \neq (R, m)$ but the encodings are identical (first $v \log_2 w$ bits of hash output).

Lemma 6.1 (E1 implies E1' or guessing)**.** *If E1 occurs, then either E1' was triggered during the game, or the adversary outputs a forgery (i, e, m^*, R^*) without having queried $H(\mathsf{PP} \| \mathsf{D}_{\mathsf{ENC}} \| e \| R^* \| m^*)$.*

[1] The ordering of the random oracle query and the signature request does not matter

Proof. Suppose E1 occurs.

Case 1: The adversary previously queried $H(\mathsf{PP}\|\mathsf{D}_{\mathsf{ENC}}\|e\|R^*\|m^*)$. Since this query returned an encoding equal to the signed encoding (from the signature request at epoch e), and $(R^*, m^*) \neq (R, m)$, the condition for E1' is satisfied. Thus E1' was triggered.

Case 2: The adversary never queried $H(\mathsf{PP}\|\mathsf{D}_{\mathsf{ENC}}\|e\|R^*\|m^*)$. Then the adversary is guessing that this unqueried input yields the same encoding as the signed message. The probability of success is at most $1/w^v$ (the probability that two independent hash outputs agree on their first $v \log_2 w$ bits). □

Corollary 6.2. $\Pr[E1] \leq \Pr[E1'] + 1/w^v$.

6.1 Compromised Node Definition

We define a node of the tree as *compromised* if its history (of queries and responses) satisfies E1'.

All descendants of *compromised* nodes are *compromised*. The root (empty history) is not *compromised*.

6.2 Transition Probabilities

Let us consider a node which is not *compromised*, and analyze the probability that one of its children becomes *compromised*.

Random Oracle Query:

For E1' to be triggered, the query should have the form $(\mathsf{PP}\|\mathsf{D}_{\mathsf{ENC}}\|e\|R'\|m')$ and the first $v \log_2 w$ bits of the hash output should match a previously signed encoding. Since there is at most one signed encoding per (signer, epoch) pair:

$$\Pr[\text{compromised from random oracle query}] \leq \frac{1}{w^v}$$

Signature Request:

Consider a signature request for signer i (with public parameter PP) at epoch e on message m.

Let $E_{i,e}$ denote the number of previous random oracle queries of the form $(\mathsf{PP}\|\mathsf{D}_{\mathsf{ENC}}\|e\|\cdot\|\cdot)$ made by the adversary, and let $E^+_{i,e}$ denote the number among these that yield **valid encodings**.

The adversary has queried $E_{i,e}$ values of R, leaving $2^r - E_{i,e}$ unqueried choices. Among the unqueried choices, the number yielding valid encodings is a random variable. Specifically, conditioned on the adversary's queries, the number of unqueried R values yielding valid encodings follows a binomial distribution, which we can bound using concentration inequalities.

By Theorem A.2 applied to the $2^r - E_{i,e}$ unqueried values (each yielding a valid encoding with probability P_{enc}), except with probability at most $1/(2^r - E_{i,e})^2 \leq 1/(2^r - q)^2$, the number of unqueried valid encodings is at least

$$(2^r - E_{i,e}) \cdot P_{\text{enc}} - 2 \cdot \sqrt{(2^r - E_{i,e}) \ln(2^r - E_{i,e})} \geq (2^r - q) \cdot P_{\text{enc}} - 2 \cdot \sqrt{r2^r} \geq P_{\text{enc}} \cdot 2^{r-1}$$

where the last inequality holds provided $q < 2^{r-2}$ and $P_{\text{enc}} \geq 8\sqrt{r/2^r}$.

We now condition on this event. The challenger samples R uniformly without replacement until finding a valid encoding. The probability that the chosen R corresponds to one of the adversary's $E^+_{i,e}$ prior valid encoding queries is:

$$\frac{E^+_{i,e}}{E^+_{i,e} + \#(\text{unqueried valid encodings})} \leq \frac{E^+_{i,e}}{E^+_{i,e} + P_{\text{enc}} \cdot 2^{r-1}} \leq \frac{E^+_{i,e}}{P_{\text{enc}} \cdot 2^{r-1}}$$

Now condition on the event that the chosen R was not previously queried. In this case, the encoding is uniform among all $P_{\text{enc}} \cdot w^v$ valid encodings. The probability that it matches one of the adversary's previous valid encoding queries at this epoch is at most $E^+_{i,e}/(P_{\text{enc}} \cdot w^v)$.

We conclude that the total transition probability to a *compromised* node satisfies:

$$\Pr[\text{compromised from signature}] \leq \frac{1}{(2^r - q)^2} + E^+_{i,e} \cdot \left[\frac{1}{P_{\text{enc}} \cdot 2^{r-1}} + \frac{1}{P_{\text{enc}} \cdot w^v}\right]$$

6.3 Final Bound

For E1' to be triggered, the final node must be *compromised*. By a union bound over all q random oracle queries and LN signature requests, we have:

$$\Pr[\text{E1'}] \leq \frac{q}{w^v} + \frac{LN}{(2^r - q)^2} + \left[\frac{1}{P_{\text{enc}} \cdot 2^{r-1}} + \frac{1}{P_{\text{enc}} \cdot w^v}\right] \cdot \sum_{i,e} \mathbb{E}(E^+_{i,e})$$

where LN is the maximum number of signature requests. Given the fact that each random oracle query can contribute at most 1 to a single $E^+_{i,e}$ (for the corresponding signer and epoch), with an independent success probability of P_{enc}, thus $\sum_{i,e} E^+_{i,e}$ is dominated by Binomial(q, P_{enc}), so $\mathbb{E}[\sum_{i,e} E^+_{i,e}] \leq q \cdot P_{\text{enc}}$.

A more formal description of this argument can be found in Theorem A.4.

By Theorem 6.2, we have $\Pr[\text{E1}] \leq \Pr[\text{E1'}] + 1/w^v$. We conclude:

Theorem 6.3 (E1 Bound)**.** *Under the constraints $q \leq 2^{r-2}$ and $P_{\text{enc}} \geq 8\sqrt{r/2^r}$:*

$$\begin{aligned}
\Pr[\textit{E1}] &\leq \Pr[\textit{E1'}] + \frac{1}{w^v} \\
&\leq \frac{q}{w^v} + \frac{LN}{(2^r - q)^2} + qP_{\text{enc}} \cdot \left[\frac{1}{P_{\text{enc}} \cdot 2^{r-1}} + \frac{1}{P_{\text{enc}} \cdot w^v}\right] + \frac{1}{w^v} \\
&\leq \frac{2q+1}{w^v} + \frac{LN}{4^{r-1}} + \frac{q}{2^{r-1}}
\end{aligned}$$

7 Event E2: WOTS Hash Chain Collision

We consider a simpler game E2' in which the adversary wins if at some point during execution, one of the following conditions is triggered:

- There exists a previous random oracle query of the form $\mathsf{query} = (\mathsf{PP} \| \mathsf{D}_{\mathsf{CHAIN}} \| e \| j \| k \| x)$ and a previous[2] signature request (i, e, m), with public parameter PP, such that, if we denote by $(\mathsf{sig}_0, \ldots, \mathsf{sig}_{v-1}) = (c_{0,x_0}, \ldots, c_{v-1,x_{v-1}})$ the corresponding WOTS signature, we have $H(\mathsf{query}) = c_{j,k+1}$, and one of the following holds:
  - E2A (hidden preimage): $k < x_j$ (the value x may or may not equal $c_{j,k}$)
  - E2B (different revealed preimage): $k \geq x_j$ and $x \neq c_{j,k}$
- E2C (premonitory preimage): There exists a previous random oracle query of the form $\mathsf{query} = (\mathsf{PP} \| \mathsf{D}_{\mathsf{CHAIN}} \| e \| j \| k \| x)$ and **no** previous corresponding signature request, such that $H(\mathsf{query}) = c_{j,k+1}$.

Lemma 7.1 (E2 implies E2' or guessing)**.** *If E2 occurs, then either E2' was triggered during the game, or the adversary outputs a forgery with a WOTS signature such that, when reconstructing the WOTS public key, one of the hash has never been queried to the random oracle (by the adversary).*

[2]The ordering of these two requests does not matter.

Proof of Theorem 7.1. Suppose E2 occurs at epoch e for signer i with public parameter PP. Let $(x_0, \ldots, x_{v-1})$ and $(\mathsf{sig}_0, \ldots, \mathsf{sig}_{v-1})$ denote the authentic encoding and WOTS signature, and let $(x_0^*, \ldots, x_{v-1}^*)$ and $(\mathsf{sig}_0^*, \ldots, \mathsf{sig}_{v-1}^*)$ denote the forged encoding and WOTS signature.

By the definition of E2, there exists an index $j \in \{0, \ldots, v-1\}$ such that $x_j^* < x_j$ (by incomparable encodings) *(case A)* or $\mathsf{sig}_j^* \neq \mathsf{sig}_j$ and $x_j^* = x_j$ (same encodings, different WOTS signature) *(case B)*.

Let's denote by $(c_{j,x_j}, \ldots, c_{j,w-1})$ and $(c^*_{j,x_j^*}, \ldots, c^*_{j,w-1})$ the hash chain values computed during public key reconstruction from the authentic and forged signatures, respectively.

- *case A*:
  - if $c_{j,x_j^*} = c_{j,x_j}$ (the adversary has found a hidden value) then either the initial query of the adversary's chain $H(\mathsf{PP}\|\mathsf{D}_{\mathsf{CHAIN}}\|e\|j\|x_j^*\|c_{j,x_j^*})$ was previously made (triggering E2A), or it's the guessing case.
  - otherwise $c_{j,x_j^*} \neq c_{j,x_j}$, but the end of the two chains matches: there must be a collision point, at index k. Again, either the query $H(\mathsf{PP}\|\mathsf{D}_{\mathsf{CHAIN}}\|e\|j\|k\|c^*_{j,k})$ was previously made (triggering either E2A or E2B, depending on $k \leq ? x_j$), or it's the guessing case.
- *case B*: There must be a collision point at index $k \geq x_j$ where $c^*_{j,k} \neq c_{j,k}$ but $c^*_{j,k+1} = c_{j,k+1}$. Again, either the query $H(\mathsf{PP}\|\mathsf{D}_{\mathsf{CHAIN}}\|e\|j\|k\|c^*_{j,k})$ was previously made (triggering E2B), or it's the guessing case.

□

Corollary 7.2. $\Pr[E2] \leq \Pr[E2'] + \frac{w}{2^n - q}$.

Proof of Theorem 7.2. In the guessing case in Theorem 7.1, there is a WOTS chain, at index j, containing a hash, at position k, that has never been queried by the adversary to the random oracle. Using Theorem 7.3, there is a probability at most $1/(2^n - q)$ that the adversary retrieves $c_{j,k}$. Otherwise, with probability $1/2^n$, after the first hash, a collision occurs with $c_{j,k+1}$. Otherwise, with probability $1/2^n$, after the second hash, a collision occurs with $c_{j,k+2}$, ..., until position $w-1$. Using the union bound, the total probability of success in the guessing case is at most: $1/(2^n - q) + (w-1)/2^n \leq w/(2^n - q)$. □

7.1 Compromised Node Definition

We define a node of the tree as *compromised* if its history satisfies E2'.

All descendants of *compromised* nodes are *compromised*. The root (empty history) is not *compromised*.

7.2 Uniformity of Hidden Values

Lemma 7.3 (Uniformity of secret chain values)**.** *At any point in the game, let $\mathcal{T}$ denote the transcript of all queries and responses observed by the adversary. For any unrevealed [3] chain value $c_{j,k}$ of signer i at epoch e, and any $x \in \{0,1\}^n \setminus [X_{i,e,j,k} \cup Y_{i,e,j,k-1}]$:*

$$\Pr\big[c_{j,k} = x \;\big|\; \mathcal{T}, \neg\mathrm{E2}'\big] = \frac{1}{|\{0,1\}^n \setminus [X_{i,e,j,k} \cup Y_{i,e,j,k-1}]|}$$

Where $X_{i,e,j,k}$ denote the set of all x such that a random oracle query of the form $(\mathsf{PP}\|\mathsf{D}_{\mathsf{CHAIN}}\|e\|j\|k\|x)$ has previously been made by the adversary, and $Y_{i,e,j,k}$ denote the set corresponding oracle outputs.

(With the convention that $Y_{i,e,j,-1} = \emptyset$)

[3] either the signature has not been requested yet, or $k < x_j$

Proof. We proceed by induction on the number of queries (random oracle queries and signature requests).

Base case: Before any interaction, the secret keys $\mathsf{sk}_j = c_{j,0}$ are sampled uniformly at random from $\{0,1\}^n$. All subsequent chain values are determined by $c_{j,k+1} = H(\mathsf{PP} \| \mathsf{D}_{\mathsf{CHAIN}} \| e \| j \| k \| c_{j,k})$. Since H is modeled as a random oracle and no queries have been made, each unrevealed chain value is uniform over $\{0,1\}^n$ from the adversary's view. The set of queried values at each position is empty, so the claim holds trivially.

Inductive step (random oracle query): Assume the lemma holds before the adversary makes a query $H(\mathsf{query})$ receiving output y.

Case 1: If query is not of the form $(\mathsf{PP} \| \mathsf{D}_{\mathsf{CHAIN}} \| e \| j \| k \| x)$ for any valid parameters, then by domain separation, this query is independent of all chain computations, and the conditional distribution of unrevealed chain values remains unchanged.

Case 2: If $\mathsf{query} = (\mathsf{PP} \| \mathsf{D}_{\mathsf{CHAIN}} \| e \| j \| k \| x)$ for signer i with public parameter PP. Consider the chain value $c_{j,k}$ for this signer at epoch e.

If $c_{j,k}$ has already been revealed, there is nothing to prove for this value. Otherwise:

- If $x = c_{j,k}$: Contradicts E2'.
- If $x \neq c_{j,k}$: By the inductive hypothesis, $c_{j,k}$ was uniform over $\{0,1\}^n \setminus [X_{i,e,j,k} \cup Y_{i,e,j,k-1}]$. Conditioned on $x \neq c_{j,k}$, $c_{j,k}$ remains uniform over $\{0,1\}^n \setminus [X_{i,e,j,k} \cup \{x\} \cup Y_{i,e,j,k-1}]$, and the induction relation is still satisfied after updating: $X_{i,e,j,k} \leftarrow X_{i,e,j,k} \cup \{x\}$.

  Now, assuming $c_{j,k+1}$ has not been revealed (otherwise nothing to prove about it), let's denote $y = H(\mathsf{query})$:

  - If $y = c_{j,k+1}$: Contradicts E2'.
  - If $y \neq c_{j,k+1}$: By the inductive hypothesis, $c_{j,k+1}$ was uniform over $\{0,1\}^n \setminus [X_{i,e,j,k+1} \cup Y_{i,e,j,k}]$. Conditioned on $y \neq c_{j,k+1}$, $c_{j,k+1}$ remains uniform over $\{0,1\}^n \setminus [X_{i,e,j,k+1} \cup Y_{i,e,j,k} \cup \{y\}]$, and the induction relation is still satisfied after updating: $Y_{i,e,j,k} \leftarrow Y_{i,e,j,k} \cup \{y\}$.

Inductive step (signature request): Assume the lemma holds before a signature request for signer i at epoch e. The challenger reveals the WOTS signature, including c_{j,x_j} for each chain j. Assume it does not trigger E2' (otherwise nothing to prove), and in particular it does not trigger E2C. We must show that for $k < x_j$, conditioning on c_{j,x_j} preserves the uniform distribution of $c_{j,k}$ over $S_k := \{0,1\}^n \setminus [X_{i,e,j,k} \cup Y_{i,e,j,k-1}]$.

Since ¬E2C holds, the adversary has never queried inputs or received outputs corresponding to intermediate chain values $c_{j,k}, c_{j,k+1}, \ldots, c_{j,x_j-1}$. For any $u \in S_k$, let $p(u)$ denote the probability (from the adversary's view) that a chain of $x_j - k$ random oracle evaluations starting from u terminates at the observed value c_{j,x_j}. By symmetry of a random function on unqueried inputs, $p(u) = p$ is identical for all $u \in S_k$.

By Bayes' theorem, using the induction hypothesis $\Pr[c_{j,k} = u \mid \mathcal{T}, \neg\text{E2}'] = 1/|S_k|$:

$$\begin{aligned}
&\Pr[c_{j,k} = u \mid c_{j,x_j}, \mathcal{T}, \neg\text{E2}'] \\
&= \frac{\Pr[c_{j,k} = u \mid \mathcal{T}, \neg\text{E2}'] \cdot \Pr[c_{j,x_j} \mid c_{j,k} = u, \mathcal{T}, \neg\text{E2}']}{\Pr[c_{j,x_j} \mid \mathcal{T}, \neg\text{E2}']} \\
&= \frac{(1/|S_k|) \cdot p}{\sum_{u' \in S_k} (1/|S_k|) \cdot p} = \frac{1}{|S_k|}
\end{aligned}$$

Since signature requests do not modify $X_{i,e,j,k}$ or $Y_{i,e,j,k-1}$, the induction property is preserved. □

7.3 Transition Probabilities

Let us consider a node which is not *compromised*, and analyze the probability that one of its children becomes *compromised*.

Random Oracle Query:

Consider a query of the form $(\mathsf{PP}\|\mathsf{D}_{\mathsf{CHAIN}}\|e\|j\|k\|x)$ for signer i with public parameter PP.

Case A: The corresponding signature (for signer i at epoch e) has not yet been requested.

Using Theorem 7.3, $c_{j,k}$ is uniform over $\{0,1\}^n \setminus [X_{i,e,j,k} \cup Y_{i,e,j,k-1}]$. With probability at most $1/(2^n - |X_{i,e,j,k}| - |Y_{i,e,j,k-1}|)$, the query hits the secret chain value: $x = c_{j,k}$ (triggering E2C and thus a transition to a *compromised* node).

If $x \neq c_{j,k}$, the output y is uniform over $\{0,1\}^n$. The probability that $y = c_{j,k+1}$ (which would trigger E2C) is $1/2^n$.

Overall:

$$\Pr[\text{E2C from query}] \leq \frac{1}{2^n - |X_{i,e,j,k}| - |Y_{i,e,j,k-1}|} + \frac{1}{2^n} \leq \frac{2}{2^n - q}$$

Case B: The signature has previously been requested, with encoding $(x_0, \ldots, x_{v-1})$.

Sub-case B1 ($k < x_j$): The chain value $c_{j,k}$ is unrevealed. Using Theorem 7.3, with probability $1/(2^n - |X_{i,e,j,k}| - |Y_{i,e,j,k-1}|)$, we have $x = c_{j,k}$, and the output equals $c_{j,k+1}$, triggering E2A.

If $x \neq c_{j,k}$, the output y is uniform over $\{0,1\}^n$. The probability that $y = c_{j,k+1}$ (triggering E2A) is $1/2^n$.

Overall:

$$\Pr[\text{E2A from query}] \leq \frac{1}{2^n - |X_{i,e,j,k}| - |Y_{i,e,j,k-1}|} + \frac{1}{2^n} \leq \frac{2}{2^n - q}$$

Sub-case B2 ($k \geq x_j$): The chain value $c_{j,k}$ has been revealed. If $x = c_{j,k}$, then E2B is not triggered by this query. If $x \neq c_{j,k}$, the output y is uniform over $\{0,1\}^n$, and the probability that $y = c_{j,k+1}$ (triggering E2B) is $1/2^n$.

Overall:

$$\Pr[\text{E2B from query}] \leq \frac{1}{2^n}$$

Since these cases are mutually exclusive (each query falls into exactly one case), we conclude:

$$\Pr[\text{compromised from query}] \leq \frac{2}{2^n - q}$$

Signature Request:

Since the node is not *compromised*, E2C has not been triggered, meaning no preimage of any $c_{j,k+1}$ has been found by the attacker for unsigned epochs. No transition to a *compromised* node occurs from a signature request.

7.4 Final Bound

Summing contributions from all q random oracle queries:

$$\Pr[\text{E2'}] \leq \frac{2q}{2^n - q}$$

By Theorem 7.2:

Theorem 7.4 (E2 Bound)**.**

$$\Pr[E2] \leq \Pr[E2'] + \frac{w}{2^n - q} \leq \frac{2q}{2^n - q} + \frac{w}{2^n - q} = \frac{2q + w}{2^n - q}$$

8 Event E3: WOTS Public Key Collision

We consider the simpler game E3', in which the adversary wins when one of its random oracle queries has the form $(\mathsf{PP}\|\mathsf{D}_{\mathsf{LEAF}}\|e\|\mathsf{pk}^*)$ and collides with the legitimate leaf hash [4] for signer i (with public parameter PP) at epoch e, with WOTS public key pk, where $\mathsf{pk}^* \neq \mathsf{pk}$.

Lemma 8.1 (E3 implies E3' or guessing)**.** *If E3 occurs, then either E3' was triggered during the game, or the adversary outputs a forgery with a WOTS public key* pk^* *such that the corresponding leaf hash* $H(\mathsf{PP}\|\mathsf{D}_{\mathsf{LEAF}}\|e\|\mathsf{pk}^*)$ *was never queried to the random oracle.*

Proof. Trivial □

Corollary 8.2. $\Pr[E3] \leq \Pr[E3'] + 1/2^n$.

Proof. By Theorem 8.1, if E3 occurs, either E3' was triggered or the adversary is guessing that an unqueried leaf hash equals the authentic one. In the guessing case, since the hash function is modeled as a random oracle, the probability that the unqueried value $H(\mathsf{PP}\|\mathsf{D}_{\mathsf{LEAF}}\|e\|\mathsf{pk}^*)$ (with $\mathsf{pk}^* \neq \mathsf{pk}$) equals the authentic leaf hash is exactly $1/2^n$. □

8.1 Compromised Node Definition

We define a node of the tree as *compromised* if its history satisfies E3'.

All descendants of *compromised* nodes are *compromised*. The root (empty history) is not *compromised*.

8.2 Transition Probabilities

Let us consider a node which is not *compromised*, and analyze the probability that one of its children becomes *compromised*.

Random Oracle Query:

Consider a query of the form $(\mathsf{PP}\|\mathsf{D}_{\mathsf{LEAF}}\|e\|\mathsf{pk}^*)$ (queries with a different form cannot trigger E3'). Let pk be the legitimate WOTS public key for signer i (with public parameter PP) at epoch e.

If $\mathsf{pk}^* = \mathsf{pk}$, then E3' cannot be triggered by this query.

If $\mathsf{pk}^* \neq \mathsf{pk}$, the output is uniform over $\{0,1\}^n$, and thus collides with the legitimate leaf hash with probability $1/2^n$:

$$\Pr[\text{compromised from query}] \leq \frac{1}{2^n}$$

Signature Request:

The game E3' is triggered only by random oracle queries, so no transition to a *compromised* node occurs from a signature request.

8.3 Final Bound

Summing over all q queries:

$$\Pr[\text{E3'}] \leq \frac{q}{2^n}$$

By Theorem 8.2:

Theorem 8.3 (E3 Bound)**.**

$$\Pr[E3] \leq \Pr[E3'] + \frac{1}{2^n} \leq \frac{q+1}{2^n}$$

[4] no need for the adversary to have already requested a signature for signer i at epoch e

9 Event E4: Merkle Tree Collision

We consider the simpler game E4', in which the adversary wins when one of its random oracle queries has the form $(\mathsf{PP}\|\mathsf{D}_{\mathsf{TREE}}\|d\|j\|A^*\|B^*)$ and collides with the corresponding legitimate internal node hash for signer i (with public parameter PP) at depth d and index j, with siblings (A, B), where $(A^*, B^*) \neq (A, B)$.

Lemma 9.1 (E4 implies E4' or guessing)**.** *If E4 occurs, then either E4' was triggered during the game, or the adversary outputs a forgery containing an authentication path such that at least one of the internal node hashes computed during verification was never queried to the random oracle.*

Proof. Trivial □

Corollary 9.2. $\Pr[E4] \leq \Pr[E4'] + h/2^n$.

Proof. By Theorem 9.1, if E4 occurs, either E4' was triggered or we are in the guessing case. In the last case, we consider a union bound over all the Merkle compression steps (at most h) that have not yet been queried. Each produces a collision with probability $1/2^n$. □

9.1 Compromised Node Definition

We define a node of the tree as *compromised* if its history satisfies E4'. All descendants of *compromised* nodes are *compromised*. The root (empty history) is not *compromised*.

9.2 Transition Probabilities

Let us consider a node which is not *compromised*, and analyze the probability that one of its children becomes *compromised*.

Random Oracle Query:

Consider a query of the form $(\mathsf{PP}\|\mathsf{D}_{\mathsf{TREE}}\|d\|j\|A^*\|B^*)$ (queries with a different form cannot trigger E4'). Let (A, B) be the legitimate siblings at that position for signer i (with public parameter PP).

If $(A, B) = (A^*, B^*)$, then E4' cannot be triggered by this query.

If $(A, B) \neq (A^*, B^*)$, the output is uniform over $\{0, 1\}^n$, and thus collides with the legitimate Merkle node hash with probability $1/2^n$:

$$\Pr[\text{compromised from query}] \leq \frac{1}{2^n}$$

Signature Request:

The game E4' is triggered only by random oracle queries, so no transition to a *compromised* node occurs from a signature request.

9.3 Final Bound

Summing over all q queries:

$$\Pr[\text{E4'}] \leq \frac{q}{2^n}$$

By Theorem 9.2:

Theorem 9.3 (E4 Bound)**.**

$$\Pr[E4] \leq \Pr[E4'] + \frac{h}{2^n} \leq \frac{q+h}{2^n}$$

10 Main Result

10.1 Without public parameter collision

Theorem 10.1 (XMSS Multi-User SUF-CMA Security without Public Parameter Collision)**.** *Let $\mathcal{A}$ be a (classical) adversary against XMSS in the multi-user SUF-CMA game, making q random oracle queries. Conditioned on the fact the all signer's public parameter are distinct (which happens with probability $1 - \Pr[PP\ collision] \geq 1 - N^2/2^{\mathsf{pp}+1}$, independently from the adversary), the probability p that $\mathcal{A}$ produces a successful forgery satisfies:*

$$p \leq \Pr[E1] + \Pr[E2] + \Pr[E3] + \Pr[E4]$$

Under the constraints $q \leq 2^{r-2}$ and $P_{\text{enc}} \geq 8\sqrt{r/2^r}$:

$$p \leq \frac{2q+1}{w^v} + \frac{LN}{4^{r-1}} + \frac{q}{2^{r-1}} + \frac{6q+2w+h+1}{2^n}$$

Proof. By the union bound over Theorem 5.1 and Theorems 6.3, 7.4, 8.3 and 9.3, using Theorem 5.2 to ensure exhaustiveness. □

10.2 Allowing Collisions Among Public Parameters

It is possible to extend the previous analysis to a scenario where public parameters may collide.

Lemma 10.2. *With N signers and pp-bit public parameters, except with probability $1/2^{2\mathsf{pp}}$, each public parameter value is shared by at most $M := 6\mathsf{pp}/(\mathsf{pp} - \log_2 N + 1)$ signers. As long as $N \leq 2^{\mathsf{pp}/2}$, we have $M \leq 12$.*

Proof. Using *Theorem A*.5. □

When multiple signers share a public parameter, the adversary can attack them simultaneously. We can adapt the previous security analysis to this scenario. The resulting bound would be the same as Theorem 10.1 with a loss of $\log_2 M$ bits of security.

There remains a subtelty: when multiple signers share the same public parameter, there may exist collisions (or revealed pre-images) among their respective signatures (either at encoding / WOTS / Merkle tree etc). Denote by E_{coll} this event. Using *Theorem A*.6 (with $k = N, n = \mathsf{pp}, x = 2Lwv/n$), $\Pr[E_{coll}] \leq 4NLwv/n$. We condition our following result on this event not occurring.

Note 1. In our current model, the signatures are revealed to the adversary "for free". A more realistic model would include each hash observation from a signature in the total number of queries q performed by the adversary. In this case, observing signatures from different signers (sharing the same public parameter) would not bring any advantage to the adversary in discovering collisions / pre-images. TODO analyze this model.

Theorem 10.3 (XMSS Multi-User SUF-CMA Security with Colliding Public Parameters)**.** *Let $\mathcal{A}$ be a (classical) adversary against XMSS in the multi-user SUF-CMA game, making q random oracle queries. Conditioned on $\neg E_{coll}$ (E_{coll} occurs with probability at most $4NLwv/n$, independently from the adversary), the probability p that $\mathcal{A}$ produces a successful forgery satisfies:*

$$p \leq \frac{1}{4^{\mathsf{pp}}} + 12 \cdot (\Pr[E1] + \Pr[E2] + \Pr[E3] + \Pr[E4])$$

Under the constraint $N \leq 2^{\mathsf{pp}/2}$.

Proof. Let's denote by C the event of collision among signer's public parameters. Let $p(\mathcal{A}|q)$ (resp. $p(\mathcal{A}|q, \neg C)$) denote the success probability of adversary $\mathcal{A}$ in the multi-user SUF-CMA game, making q random oracle queries (resp. conditioned on $\neg C$).

Theorem 10.1 gives a bound on $p(\mathcal{A}|q, \neg C)$.

Let $\mathcal{A}_0$ be an adversary. Consider the following derived algorithm $\mathcal{A}_1$: at the start of the game, after receiving the N XMSS public keys, which we assume to have distinct public parameters, $\mathcal{A}_1$ creates $N(M-1)$ "dumy" XMSS key pairs (reusing each public parameter $M-1$ times). Then, $\mathcal{A}_1$ run $\mathcal{A}_0$, with the NM XMSS public keys as input. Whenever $\mathcal{A}_0$ requests a signature from a "real" signer, or peforms a random oracle query, $\mathcal{A}_1$ forwards it to the challenger. Whenever $\mathcal{A}_0$ requests a signature from a "dumy" signer, $\mathcal{A}_1$ can compute it itself (up to some additional random oracle queries for the encoding). The key idea: A_0 cannot distinguish between real and dumy signers. In the end, $\mathcal{A}_1$ outputs the same forgery as $\mathcal{A}_0$. In case of success of $\mathcal{A}_0$, there is $1/M$ probability that the forgery involves a "real" signer. Let's denote by q_0 (resp. q_1) the number of random oracle queries performed by $\mathcal{A}_0$ (resp. $\mathcal{A}_1$). We thus have:

$$p(\mathcal{A}_0|q_0) = M \cdot p(\mathcal{A}_1|q_1, \neg C)$$

Overall $\mathcal{A}_1$ performs on average $q_1 - q_0 = N(M-1)L((w-1)v + 2 + 1/P_{\text{enc}})$ additional random oracle queries compared to $\mathcal{A}_0$.

The conclusion follows naturally. This $q_1 - q_0$ still causes some loss of security, but is negligible for practical parameters. We do not make it explicit in the statement, because possible a tighter analysis would avoid this loss, following *Theorem* 10.1's proof. TODO should we instead use the sightly looser but simpler bound following the reduction proof? Probably □

11 Concrete Parameter Selection

We use the lifetime: $L = 2^{32}$.

We present concrete bounds when the hash function is derived from Poseidon2 [4] over 16 field elements in $\mathbb{F}_p$, with $p = 2^{31} - 2^{24} + 1$ (koala-bear prime). In this setting, we conservatively assume that hashing the WOTS public key requires v hashes (even though we can do better, but this would require further explanation (TODO)). Finally, we assume the encoding can be computed in 3 hashes.

11.1 147-bit security, 2^{50} signers

(Using *Theorem* 10.3)

- Hash output: $n \geq 154$ bits
- Public parameter: $\mathsf{pp} \geq 120$ bits
- WOTS parameters: $v \cdot \log_2 w \geq 160$ bits
- Randomness: $r \geq 154$ bits

Using an algebraic hash function over koala-bear: **we can use hash digests of 5 field elements**, a randomness of 5 field elements, and public parameters of 4 field elements. 2 field elements are used for the tweak (domain separation and hash position).

Here are two example parameter sets:

- **Lightweight**
  - $w = 16$, $v = 40$

  - target sum $t = 360$ (9.2 bits of grinding at signing to find a valid randomness), 240 hashes per WOTS verification
  - $3 + 240 + 40 + 32 = 315$ hashes per XMSS verification
  - Signature size: 1.42 KiB

- **Fewer hashes**
  - $w = 8$, $v = 54$
  - target sum $t = 225$ (8.7 bits of grinding at signing to find a valid randomness), 153 hashes per WOTS verification
  - $3 + 153 + 54 + 32 = 242$ hashes per XMSS verification
  - Signature size: 1.69 KiB

11.2 128-bit security, 2^{50} signers

(Using *Theorem* 10.3)

- Hash output: $n \geq 135$ bits
- Public parameter: $\mathsf{pp} \geq 100$ bits
- WOTS parameters: $v \cdot \log_2 w \geq 135$ bits
- Randomness: $r \geq 135$ bits

Using an algebraic hash function over koala-bear: **we can use hash digests of 5 field elements**, a randomness of 5 field elements, and public parameters of 4 field elements. 2 field elements are used for the tweak (domain separation and hash position).

Here are two example parameter sets:

- **Lightweight**
  - $w = 16$, $v = 34$
  - target sum $t = 310$ (9 bits of grinding at signing to find a valid randomness), 200 hashes per WOTS verification
  - $3 + 200 + 34 + 32 = 269$ hashes per XMSS verification
  - Signature size: 1.27 KiB

- **Fewer hashes**
  - $w = 8$, $v = 45$
  - target sum $t = 190$ (8.5 bits of grinding at signing to find a valid randomness), 125 hashes per WOTS verification
  - $3 + 125 + 45 + 32 = 205$ hashes per XMSS verification
  - Signature size: 1.48 KiB

11.3 121-bit security, 2^{30} signers

(Using *Theorem* 10.1)

- Hash output: $n \geq 123.9$ bits
- Public parameter: $\mathsf{pp} \geq 185$ bits
- WOTS parameters: $v \cdot \log_2 w \geq 124$ bits
- Randomness: $r \geq 127$ bits

Using an algebraic hash function over koala-bear: **we can use hash digests of 4 field elements**, a randomness of 5 field elements, and public parameters of 6 field elements. 2 field elements are used for the tweak (domain separation and hash position).

Here are two example parameter sets:

- **Lightweight**
  - $w = 16$, $v = 31$
  - target sum $t = 285$ (9 bits of grinding at signing to find a valid randomness), 180 hashes per WOTS verification
  - $3 + 180 + 31 + 32 = 246$ hashes per XMSS verification
  - Signature size: 0.98 KiB
- **Fewer hashes**
  - $w = 8$, $v = 42$
  - target sum $t = 180$ (8.8 bits of grinding at signing to find a valid randomness), 114 hashes per WOTS verification
  - $3 + 114 + 42 + 32 = 191$ hashes per XMSS verification
  - Signature size: 1.14 KiB

12 TODO

- Full proof for *Theorem* 10.3
- quantum adversary
- explicit construction of the encoding (currently treated as a black box). Proof of security even in case of non-uniform encoding (key property: probability of the most likely encoding $\leq 1/2^{\text{security level}}$)
- explicit construction of the compression of the WOTS public key (currently treated as a black box).
- Even tighter bounds? We can likely gain 2 or 3 bits of security by refining some of the union bounds, as well as increasing the number of signers.

A Annex: Technical Results

A.1 Birthday Bound

Lemma A.1 (Birthday Bound)**.** *If k values are sampled independently and uniformly at random from a set of size n, the probability that at least two values are equal is at most $k^2/(2n)$.*

Proof. Let $X_1, \ldots, X_k$ be the sampled values. By the union bound over all $\binom{k}{2}$ pairs:

$$\Pr[\exists i \neq j : X_i = X_j] \leq \sum_{1 \leq i < j \leq k} \Pr[X_i = X_j] = \binom{k}{2} \cdot \frac{1}{n} = \frac{k(k-1)}{2n} \leq \frac{k^2}{2n}. \qquad \square$$

A.2 Binomial Concentration

Lemma A.2 (Binomial Concentration)**.** *Let $X \sim \text{Binomial}(n, p)$. For any $\delta > 0$:*

$$\Pr\left(|X - np| \geq \delta\right) \leq 2 \exp\left(-\frac{2\delta^2}{n}\right)$$

In particular, with probability at least $1 - 1/n^2$:

$$|X - np| \leq \sqrt{\frac{n \ln(2n^2)}{2}} \leq 2\sqrt{n \ln n}$$

Proof. Write $X = \sum_{i=1}^{n} X_i$ where $X_1, \ldots, X_n$ are independent Bernoulli(p) random variables.

Hoeffding's inequality states that for independent random variables $Y_i \in [a_i, b_i]$:

$$\Pr\left(\left|\sum_{i=1}^{n} Y_i - \mathbb{E}\left[\sum_{i=1}^{n} Y_i\right]\right| \geq t\right) \leq 2 \exp\left(-\frac{2t^2}{\sum_{i=1}^{n}(b_i - a_i)^2}\right)$$

For Bernoulli random variables with $a_i = 0$ and $b_i = 1$:

$$\Pr(|X - np| \geq t) \leq 2 \exp\left(-\frac{2t^2}{n}\right)$$

Setting the right-hand side equal to ε and solving:

$$2 \exp\left(-\frac{2t^2}{n}\right) = \varepsilon \implies t = \sqrt{\frac{n \ln(2/\varepsilon)}{2}}$$

The stated bounds follow by substitution. Specifically, setting $\varepsilon = 1/n^2$:

$$t = \sqrt{\frac{n \ln(2n^2)}{2}} = \sqrt{\frac{n(\ln 2 + 2 \ln n)}{2}} \leq \sqrt{\frac{n \cdot 3 \ln n}{2}} \leq 2\sqrt{n \ln n}$$

for $n \geq 3$. $\square$

Here is a cleaner version:

A.3 Probabilistic Compromise Tree

We formalize the argument used the draw the final bound for E1, c.f. Section 6.3.

Definition A.3 (Probabilistic Compromise Tree)**.** Consider a rooted tree with the following properties:

- The root is initially *not compromised*.

- Each node is of one of two types: **type 1** or **type 2**.
- Along every path from root to leaf, there are exactly q nodes of type 1 and exactly n nodes of type 2.
- Each type 2 node is labeled by an integer in $\{0, 1, \ldots, n-1\}$. Along every path from root to leaf, each label in $\{0, 1, \ldots, n-1\}$ appears exactly once.
- We maintain n counters $X_0, X_1, \ldots, X_{n-1}$, all initialized to 0 at the start of any traversal.

The traversal dynamics from root to leaf are as follows:

- **Type 1 node:** If the current node is not already compromised:

  1. With probability x, all children (and hence all descendants) become compromised.

  Additionally, with probability p, the counter X_i is incremented by 1, where $i \in \{0, \ldots, n-1\}$ is a fixed index inscribed on the node.

- **Type 2 node:** Let i be the label of this node. If the current node is not already compromised:

  1. With probability $y + z \cdot X_i$, all children (and hence all descendants) become compromised.

Here $x, y, z, p \in [0, 1]$ are fixed parameters common to all nodes. Once a node becomes compromised, all its descendants are automatically compromised.

Lemma A.4 (Upper Bound on Compromise Probability)**.** *The probability that a random traversal from root to leaf becomes compromised at some point along the path satisfies:*

$$\boxed{P(\text{path is compromised}) \leq qx + ny + pqz}$$

Proof. For each node v on the path, let T_v be the indicator that v triggers compromise, regardless of whether the path is already compromised. By the union bound,

$$P(\text{path is compromised}) \leq \sum_v \mathbb{E}[T_v].$$

Type 1 contribution. Each of the q type 1 nodes triggers with probability x, contributing qx.

Type 2 contribution. The type 2 node labeled i triggers with probability $y + zX_i$, where X_i is the counter value upon arrival. Summing over all n labels:

$$\sum_{i=0}^{n-1} \mathbb{E}[y + zX_i] = ny + z \sum_{i=0}^{n-1} \mathbb{E}[X_i].$$

Now, $\sum_i X_i$ counts the total number of increments that occurred before reaching each respective type 2 node. Each of the q type 1 nodes increments some counter with probability p, and each such increment is counted in $\sum_i X_i$ only if it occurs before the corresponding type 2 node. Hence $\sum_i \mathbb{E}[X_i] \leq pq$.

Conclusion. Adding both contributions: $qx + ny + pqz$. □

A.4 Balls-into-Bins

Theorem A.5 (Balls-into-Bins Maximum Load)**.** *Let $k, n \in \mathbb{N}$ with $1 \le k \le n$. Let X be the maximum load when k balls are thrown independently and uniformly at random into n bins. Then:*

$$\boxed{\mathbb{P}\left(X < \frac{6\log n}{\log(n/k)+1}\right) \ge 1 - \frac{1}{n^2}}$$

Proof. We proceed in three steps.

Step 1: Single-bin tail bound.

Let X_i denote the number of balls in bin i. For any integer $t \ge 1$:

$$\Pr(X_i \ge t) \le \binom{k}{t}\left(\frac{1}{n}\right)^t \le \frac{k^t}{t!}\cdot\frac{1}{n^t} \le \left(\frac{ek}{tn}\right)^t$$

where we used $t! \ge (t/e)^t$.

Step 2: Union bound over bins.

By the union bound:

$$\Pr(X \ge t) \le n \cdot \left(\frac{ek}{tn}\right)^t$$

Step 3: Choosing the threshold.

We want $n \cdot (ek/(tn))^t \le 1/n^2$, equivalently:

$$t \log \frac{tn}{ek} \ge 3\log n.$$

Set $t = \dfrac{6\log n}{\log(n/k)+1}$. We have:

$$\frac{tn}{ek} = \frac{6\log n}{e\big(\log(n/k)+1\big)} \cdot \frac{n}{k}.$$

Let $\alpha = \log(n/k) \ge 0$. Then:

$$\frac{tn}{ek} = \frac{6\log n}{e(\alpha+1)} \cdot e^{\alpha}.$$

Taking logarithms:

$$\log \frac{tn}{ek} = \alpha + \log(6\log n) - \log(\alpha+1) - 1.$$

For $\alpha \ge 0$: $\log(\alpha+1) \le \alpha/2 + \log 2$

Therefore:

$$\log \frac{tn}{ek} \ge \alpha - \frac{\alpha}{2} - \log 2 - 1 + \log(6\log n) = \frac{\alpha}{2} + \underbrace{\log(6\log n) - 1 - \log 2}_{\ge 1/2 \text{ for } n \ge 4}.$$

Hence for $n \ge 4$:

$$\log \frac{tn}{ek} \ge \frac{\alpha+1}{2} = \frac{\log(n/k)+1}{2}.$$

Conclusion:

$$t \log \frac{tn}{ek} \ge \frac{6\log n}{\log(n/k)+1} \cdot \frac{\log(n/k)+1}{2} = 3\log n. \qquad \square$$

And finally:

$$\left(\frac{ek}{tn}\right)^t \le \frac{1}{n^3} \implies \Pr(X \ge t) \le \frac{1}{n^2}.$$

Lemma A.6 (Balls-into-Bins, Quadratic Triggering)**.** *Let $k, n \in \mathbb{N}$ with $1 \le k \le n$, and let $x > 0$. Suppose k balls are thrown independently and uniformly at random into n bins. If each bin containing i balls independently triggers event E with probability $i^2 x$, then:*

$$\boxed{\mathbb{P}(E \text{ is triggered in at least one bin}) \le 2xk}$$

Proof. **Step 1: Union bound.**

Let B_j denote the number of balls in bin j. By the union bound:

$$\mathbb{P}(\text{at least one trigger}) \le \sum_{j=1}^{n} \mathbb{P}(\text{bin } j \text{ triggers } E) = \sum_{j=1}^{n} \mathbb{E}[B_j^2] \cdot x = x \cdot \mathbb{E}\left[\sum_{j=1}^{n} B_j^2\right].$$

Step 2: Expected sum of squares.

Let $Y_\ell \in \{1, \dots, n\}$ denote the bin chosen by ball ℓ. The number of balls in bin j is $B_j = \sum_{\ell=1}^{k} \mathbf{1}_{Y_\ell = j}$. Expanding the square:

$$B_j^2 = \left(\sum_{\ell=1}^{k} \mathbf{1}_{Y_\ell = j}\right)^2 = \sum_{\ell=1}^{k} \sum_{m=1}^{k} \mathbf{1}_{Y_\ell = j} \cdot \mathbf{1}_{Y_m = j}.$$

Summing over all bins and exchanging the order of summation:

$$\sum_{j=1}^{n} B_j^2 = \sum_{\ell=1}^{k} \sum_{m=1}^{k} \sum_{j=1}^{n} \mathbf{1}_{Y_\ell = j} \cdot \mathbf{1}_{Y_m = j}.$$

The inner sum equals 1 if and only if $Y_\ell = Y_m$ (i.e., balls ℓ and m land in the same bin), and 0 otherwise:

$$\sum_{j=1}^{n} \mathbf{1}_{Y_\ell = j} \cdot \mathbf{1}_{Y_m = j} = \mathbf{1}_{Y_\ell = Y_m}.$$

Therefore:

$$\sum_{j=1}^{n} B_j^2 = \sum_{\ell=1}^{k} \sum_{m=1}^{k} \mathbf{1}_{Y_\ell = Y_m} = \underbrace{\sum_{\ell=1}^{k} \mathbf{1}_{Y_\ell = Y_\ell}}_{=k} + \sum_{\ell \ne m} \mathbf{1}_{Y_\ell = Y_m}.$$

Taking expectations and using that Y_ℓ and Y_m are independent for $\ell \ne m$:

$$\mathbb{E}\left[\sum_{j=1}^{n} B_j^2\right] = k + \sum_{\ell \ne m} \mathbb{P}(Y_\ell = Y_m) = k + k(k-1) \cdot \frac{1}{n}.$$

We conlude using $(k-1)/n \le 1$. □

References

[1] J. Drake, D. Khovratovich, M. Kudinov, and B. Wagner, "Hash-based multi-signatures for post-quantum ethereum," Cryptology ePrint Archive, Paper 2025/055, 2025. [Online]. Available: https://eprint.iacr.org/2025/055

[2] D. Khovratovich, M. Kudinov, and B. Wagner, "At the top of the hypercube – better size-time tradeoffs for hash-based signatures," Cryptology ePrint Archive, Paper 2025/889, 2025. [Online]. Available: https://eprint.iacr.org/2025/889

[3] J. Drake, D. Khovratovich, M. Kudinov, and B. Wagner, "Technical note: LeanSig for post-quantum ethereum," Cryptology ePrint Archive, Paper 2025/1332, 2025. [Online]. Available: https://eprint.iacr.org/2025/1332

[4] L. Grassi, D. Khovratovich, and M. Schofnegger, "Poseidon2: A faster version of the poseidon hash function," Cryptology ePrint Archive, Paper 2023/323, 2023. [Online]. Available: https://eprint.iacr.org/2023/323